AI-Powered Legal Document Analyser

A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE DEGREE OF

Master of Technology (Online)
IN
Data Science and Business Analytics

BY

Mukesh Kumar Yadav



Faculty of Engineering
Indian Institute of Science
Bangalore – 560 012 (INDIA)

June, 2026

Declaration of Originality

I, **Mukesh Kumar Yadav**, with SR No. **24481** hereby declare that the material presented in the project report titled

AI-Powered Legal Document Analyser

represents original work carried out by me at **Wipro Ltd** as part of the project credit requirements of the Master of Technology (Online) degree in Data Science at the Indian Institute of Science, between **June-2024** to **June-2026**.

With my signature, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.

Date: _____ Student Signature: _____

In our capacities as internal project guide and faculty mentor of the above-mentioned work, we certify that the above statements are true to the best of our knowledge, and we have carried out due diligence to ensure the originality of the report.

Internal Guide Name: _____ **Internal Guide Signature:** _____

Organisation: _____

Faculty Mentor Name: _____ **Faculty Mentor Signature:** _____

Department: _____

© Mukesh Kumar Yadav
June, 2026
All rights reserved

Acknowledgements

I would like to thank Prof. Pandarasamy Arjunan, my faculty mentor at IISc, for his support throughout this project. He helped me figure out the right approach to take, gave useful feedback at every stage, and pointed me in the right direction whenever I was stuck on the implementation. The overall design of this project took shape largely because of his guidance.

I also want to thank Prof. Deepak Subramani for his help with the overall process of doing the project and writing this report. He guided me on how to structure the work, what to focus on in the evaluation, and how to present the findings clearly. His advice made the whole process much smoother.

I am grateful to Mr. Manjunath Ramachandra Iyer, my company mentor, for his support during the design and implementation of the system. He gave practical advice on technical decisions, helped me think through the architecture, and guided me through the challenges that came up during development.

Finally, I would like to thank my family and friends for their patience and support throughout the M.Tech programme.

Abstract

Legal documents like contracts and agreements are very important but most people find them hard to understand because of the complex language and long clauses. Things like auto-renewal traps, one-sided liability, and penalty clauses are easy to miss if you are not a legal expert. This project tries to solve that problem by building **LexAI**, a web-based AI platform that can analyse legal documents and give users a clear, structured breakdown of what the document says and where the risks are.

The system works by letting users upload a PDF document, after which a set of AI agents analyses it from different angles — contract review, legal research, risk assessment, compliance check, and a custom query option. Each agent is built using LangChain and uses retrieval-augmented generation (RAG) to search through the actual uploaded document before giving any output. This makes sure the analysis is based on what is actually in the document, not on what the model guesses. A Team Lead agent then combines all the individual outputs into one final report.

On the technical side, the system has a React frontend talking to a FastAPI backend. The backend is split into separate modules for different tasks — document processing, chat, prompt management, PDF export, analytics, and so on. Azure OpenAI GPT-4 is used for generating the analysis and Ada-002 is used for creating the embeddings. Qdrant is used as the vector store. All the main settings like chunk size, session timeout, and page size are stored in a single `config.properties` file so they can be changed without touching the code.

Apart from the main analysis features, the platform also has a chat feature called **Talk2Docs** that lets users ask follow-up questions about the document. There is also an analysis history section where past analyses are saved with pagination, a PDF export option, an admin panel for managing users and viewing usage analytics, and a session timeout feature that logs the user out after a period of inactivity.

Testing was done using a sample SaaS agreement that had several tricky clauses built into it. The system correctly identified all the important risk clauses like the asymmetric liability cap, the compound interest clause, and the auto-renewal trap, and it gave the correct clause number for each finding. The platform is meant for students, freelancers, and small businesses who need to understand a legal document but do not have access to a lawyer. Every output includes a note that it is not legal advice.

Contents

Acknowledgements i

Abstract ii

List of Figures v

List of Tables vi

List of Algorithms vii

Acronyms, Abbreviations and Symbols viii

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Scope . . . 1
- 1.3 Objectives . . . 2

2 Literature Review 3
- 2.1 Legal Language Processing and Document Understanding . . . 3
- 2.2 Large Language Models for Legal Reasoning . . . 3
- 2.3 Retrieval-Augmented Generation . . . 3
- 2.4 Multi-Agent Systems and the ReAct Framework . . . 4
- 2.5 Existing Legal AI Tools and Their Limitations . . . 4

3 Methodology and Implementation 5
- 3.1 Technology Stack . . . 6
- 3.2 System Architecture . . . 6
  - 3.2.1 Backend Module Structure . . . 7
  - 3.2.2 Frontend Component Structure . . . 7
- 3.3 Document Ingestion and Chunking . . . 7
- 3.4 Embedding and Vector Storage . . . 8
- 3.5 Multi-Agent Analysis Framework . . . 8
- 3.6 Prompts Architecture . . . 9
- 3.7 Talk2Docs — Conversational Document Chat . . . 10
- 3.8 Session Timeout and Configuration Management . . . 11
- 3.9 API Endpoints . . . 13
- 3.10 Database Schema . . . 13
- 3.11 Configuration File . . . 13
- 3.12 Authentication and Security . . . 14
- 3.13 Data Flow . . . 14
  - 3.13.1 Document Upload . . . 14

3.13.2 Analysis Pipeline . . . 14
3.13.3 Talk2Docs Chat . . . 15

4 Results and Discussions **16**
4.1 Features Implemented . . . 16
4.1.1 Five Analysis Modes . . . 16
4.1.2 Document Library with Upload/Library Tabs . . . 16
4.1.3 Collapsible Sidebar with Rail Mode . . . 17
4.1.4 Paginated Analysis History . . . 17
4.1.5 Admin Panel and Analytics . . . 18
4.1.6 Branded PDF Export . . . 18
4.2 Evaluation . . . 19
4.2.1 Test Document . . . 19
4.2.2 Qualitative Observations . . . 19
4.2.3 Limitations . . . 21

5 Conclusions **22**
5.1 Challenges and Learnings . . . 22
5.1.1 Multi-Agent Memory Isolation . . . 22
5.1.2 Consistent Agent Output Format . . . 22
5.1.3 Backend Modularisation . . . 22
5.1.4 Session Timeout Coordination . . . 22
5.2 Security, Ethics and Compliance . . . 23
5.3 Future Work . . . 23
5.4 Conclusion . . . 23

List of Figures

3.1 Full application dashboard showing sidebar, analysis panel, and Talk2Docs button 5
3.2 Sidebar Upload tab showing drag-and-drop zone, uploaded file list, and chunk count 8
3.3 Analysis type selection cards and Run Analysis button 9
3.4 Multi-agent report output with markdown-rendered sections and clause citations 9
3.5 Talk2Docs floating button and chat drawer with active multi-turn conversation thread 11
3.6 Session timeout warning banner with live countdown and Extend Session / Log Out buttons 12

4.1 Sidebar Upload tab and Library tab showing active session files and all-time document history 17
4.2 Full sidebar state vs collapsed rail state showing status indicator and expand button 17
4.3 Admin analytics dashboard showing analysis type breakdown and daily activity trend 18
4.4 Exported PDF report showing cover page and analysis content 18
4.5 Test contract PDF uploaded to LexAI alongside the generated analysis output . 19
4.6 Risk Assessment output showing flagged clauses with severity and clause citations 20
4.7 Talk2Docs answering a question with specific clause citation 21

List of Tables

3.1 LexAI technology stack 6
3.2 Backend module responsibilities 7
3.3 Frontend component responsibilities 7
3.4 Multi-agent pipeline — roles and analysis type assignments 8
3.5 LexAI REST API endpoints 13
3.6 `config.properties` keys and their effects 13

List of Algorithms

No explicit algorithms beyond standard library implementations are used in this project. The core inference logic is delegated entirely to the LangChain ReAct agent loop and the Azure OpenAI GPT-4 API.

Acronyms, Abbreviations and Symbols

Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
GPT	Generative Pre-trained Transformer
HTTP	Hypertext Transfer Protocol
IISc	Indian Institute of Science
JSON	JavaScript Object Notation
LLM	Large Language Model
NDA	Non-Disclosure Agreement
NLI	Natural Language Inference
NLP	Natural Language Processing
ORM	Object-Relational Mapping
PDF	Portable Document Format
Q&A	Question and Answer
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SaaS	Software as a Service
SQL	Structured Query Language
UI	User Interface
UUID	Universally Unique Identifier

Abbreviations

et al.	and others
e.g.	for example (*exempli gratia*)
i.e.	that is (*id est*)
vs.	versus

Symbols

k	Number of top chunks returned per similarity search
d	Dimensionality of the embedding vector space (1536 for Ada-002)

Chapter 1

Introduction

Most people at some point have to deal with legal documents like employment contracts, rental agreements, or service agreements. These documents are often many pages long and written in complicated legal language that is hard to follow without legal training. Important things like automatic renewal clauses, liability limits, or penalty charges are often buried deep in the document and easy to miss.

With the growth of large language models (LLMs), it has become possible to use AI to help with understanding these documents. But just feeding a full contract into an LLM does not work well. Two problems come up: the model sometimes makes up things that are not actually in the document (called hallucinations), and the output is usually a general summary rather than a structured analysis. For legal documents, you need something more specific — ideally an analysis that looks at each clause and says exactly what the risk is and where in the document it comes from.

This project, called **LexAI**, tries to solve these two problems. It uses retrieval-augmented generation (RAG) so that the AI only talks about what is actually in the uploaded document. It also uses multiple agents, where each agent has a specific job — for example, one agent does the contract review while another does the risk assessment. This way the output is more structured and covers more ground.

1.1 Motivation

The main reason I chose this project is that there is a real gap between what most existing tools do (which is just summarise a document) and what someone actually needs when reviewing a contract (which is a detailed, clause-level analysis with specific citations). I also wanted to build a complete product that someone could actually use, not just a script or a notebook. That is why I built a proper frontend, added user login, history, PDF export, and other features that make the system usable end-to-end.

1.2 Scope

LexAI is mainly targeted at students, people early in their careers, and small businesses who need to understand a legal document but cannot afford to consult a lawyer every time. The system is not a replacement for legal advice, and this is made clear in every output it generates. It works with PDF documents and supports five types of analysis: contract review, legal research, risk assessment, compliance check, and a custom question option.

1.3 Objectives

The objectives set at the start of the project were:

1. Design and implement a complete full-stack legal document analysis platform, from PDF upload to structured report output and PDF export.
2. Build a multi-agent AI system where each agent has a distinct legal persona, a structured system prompt, and access to a document retrieval tool.
3. Implement RAG using a vector database so all AI responses are grounded in the uploaded document, not general model knowledge.
4. Support five analysis types: Contract Review, Legal Research, Risk Assessment, Compliance Check, and Custom Query.
5. Build a persistent, user-specific analysis history system with pagination and PDF export.
6. Implement a conversational chatbot (Talk2Docs) for natural multi-turn Q&A over uploaded documents.
7. Manage all runtime settings through a central `config.properties` file so the system is tunable without code changes.
8. Implement session security with configurable auto-logout on inactivity.
9. Build an admin panel with user management CRUD and platform analytics.
10. Produce a professional, usable frontend that a non-technical user can navigate without guidance.

Chapter 2

Literature Review

2.1 Legal Language Processing and Document Understanding

Legal text is a very different kind of writing compared to normal language. Contracts and legal documents tend to use long sentences, lots of defined terms, and complex clause structures. Because of this, early NLP approaches to legal documents used rule-based systems where experts wrote specific rules to extract information from contracts [9]. The problem with this was that rules written for one type of contract or one country's laws usually did not work well for another.

Things improved a lot when large labelled datasets for legal NLP became available. The CUAD dataset [4] has over 13,000 annotations from legal experts across 510 commercial contracts. It covers 41 types of clauses including things like change-of-control clauses, liquidated damages, and non-compete provisions. The ContractNLI dataset [5] set up the task of checking whether a given statement is true or false based on a contract, which is useful for automated contract review. LexGLUE [6] brought together seven legal NLP datasets from different countries into one benchmark for comparing models. These datasets also showed that models trained specifically on legal text, like Legal-BERT [7], do better than general models on legal tasks, especially on longer documents.

2.2 Large Language Models for Legal Reasoning

In recent years, LLMs like GPT-4 have gotten much better at handling legal tasks. For example, GPT-4 scored in the top 10% on the US Bar Exam [8], which shows that these models have a reasonable understanding of law. This has made people look into using them for tasks like contract review and legal question answering.

However, using LLMs directly for legal document analysis has two main problems. The first is *hallucination*, where the model generates something that sounds correct but is not actually in the document. In a legal context this is dangerous because a user might act on incorrect information. The second problem is that LLMs tend to give vague summaries rather than specific clause-level findings, which is not very useful for someone who needs to understand exactly what a contract says. Training the model more on legal data can help somewhat, but it is expensive and requires a lot of labelled data. A better approach in many cases is to use retrieval augmentation, which forces the model to base its answers on what is actually in the document.

2.3 Retrieval-Augmented Generation

RAG was introduced by Lewis et al. [1] as a way to combine a language model with a retrieval system. Instead of relying only on what the model has memorised during training, it first searches

a document store for relevant passages and then uses those passages to generate the answer. This reduces hallucinations because the model is working with actual text from the document rather than trying to recall facts from its training data.

For legal documents specifically, RAG works well for a few reasons. First, the document being analysed is always available, so there is no need to rely on the model's memory at all. Second, because every answer is based on retrieved chunks, you can point back to the exact part of the document that supports a finding. Third, vector search handles the fact that legal language is not always consistent — the same obligation might be described as "payment terms" in one contract and "remuneration schedule" in another, but a vector search will still find both because the meaning is similar [10].

In practice, this means splitting the document into chunks, converting each chunk into a vector using an embedding model, and storing those vectors in a vector database like Qdrant [13]. When a query comes in, the system finds the most similar chunks and gives them to the language model as context. The size of the chunks and how much they overlap matters quite a bit — if the chunks are too small you lose context, and if there is no overlap then a sentence that falls at a boundary will be cut off.

2.4 Multi-Agent Systems and the ReAct Framework

One problem with using a single LLM prompt for complex tasks is that it tries to do everything at once, which often leads to shallow output. A better approach is to break the task into smaller parts and have different agents handle each part. This is the idea behind multi-agent systems.

The ReAct framework [2] is one approach to building agents that can both think and take actions. An agent using ReAct alternates between reasoning about what it needs to do next and actually calling a tool (like a search function) to get information. LangChain uses this as the basis for its agent implementation. Another related idea is chain-of-thought prompting [3], where you ask the model to show its reasoning step by step before giving a final answer. This has been shown to improve accuracy on tasks that need multi-step thinking.

For LexAI, these ideas were used to build specialist agents where each agent has a specific role. For example, one agent is focused only on risk assessment while another focuses on checking compliance. Each agent has its own system prompt that tells it exactly what to do and what format to use for the output. A final Team Lead agent then takes all the individual outputs and combines them into one report. This is similar to how a law firm might assign different parts of a contract review to different people based on their expertise.

2.5 Existing Legal AI Tools and Their Limitations

There are already some tools available that use AI for legal document review. Harvey AI and Spellbook are two examples that use GPT-4 for legal drafting and review. However, these tools work more like a chatbot — they let you ask questions about a document but do not give a structured multi-perspective output. Kira Systems is a more specialised tool for contract review, but it needs to be trained on the company's own documents before it becomes useful, which makes it expensive and slow to set up. ContractPodAi and Ironclad are more focused on managing the lifecycle of contracts rather than doing deep analysis.

Looking at these tools, the main gaps I identified are: none of them give a structured output that covers risk, compliance, and legal research all in one place; most do not show exactly which clause in the document supports each finding; and they are generally too expensive for individuals or small businesses. LexAI was designed to address all three of these gaps directly.

Chapter 3

Methodology and Implementation

LexAI is a web application where the frontend and backend are completely separate. The React frontend talks to the FastAPI backend through HTTP API calls. The backend is split into several Python modules, each responsible for one part of the system. The AI services (Azure OpenAI and Qdrant) are treated as external dependencies that the backend connects to.

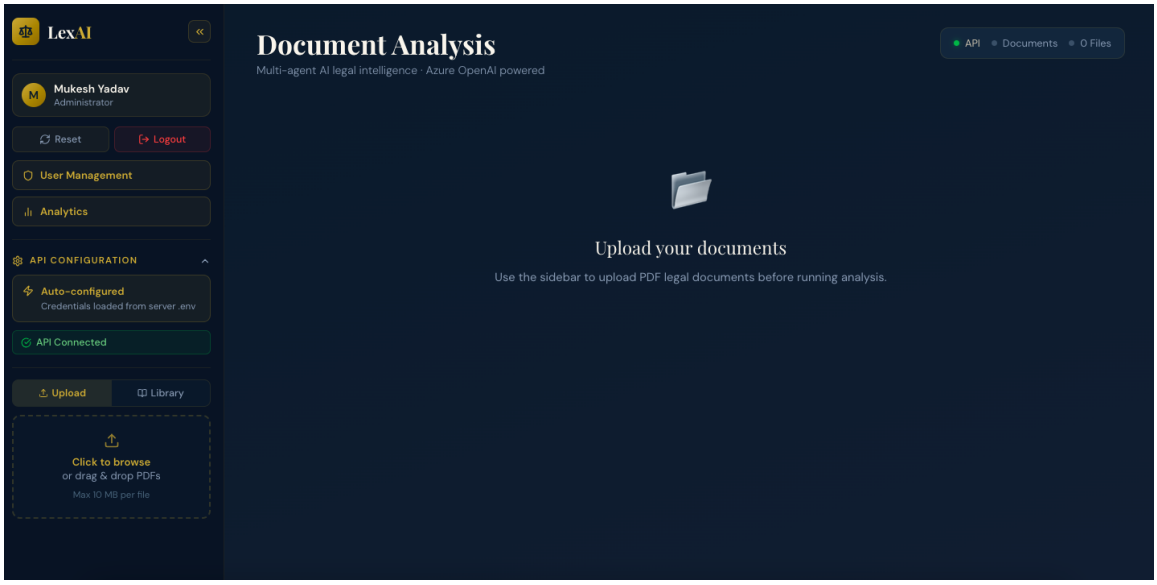


Figure 3.1: Full application dashboard showing sidebar, analysis panel, and Talk2Docs button

3.1 Technology Stack

Layer	Technology	Role
Frontend	React 18	UI, routing, state management, component rendering
HTTP Client	Axios	API calls from browser to backend
Backend API	FastAPI (Python)	REST endpoints, request validation, routing
AI Orchestration	LangChain	Agent setup, tool use, memory, ReAct reasoning loop
LLM	Azure OpenAI GPT-4	Natural language generation and legal reasoning
Embeddings	Azure Ada-002	Semantic vector representation of text chunks
Vector Store	Qdrant (in-memory)	Cosine similarity search over document chunks
PDF Parsing	PyMuPDF	Text extraction from uploaded PDF files
Text Splitting	LangChain TextSplitter	Chunking with overlap for contextual continuity
Database	SQLite + SQLAlchemy	Users, analysis history, document library
Authentication	bcrypt	Password hashing and login state
PDF Export	ReportLab	Branded analysis report generation
Markdown Render	ReactMarkdown	Renders formatted AI responses in the UI
Configuration	config.properties	Centralised runtime settings for the whole platform

Table 3.1: LexAI technology stack

3.2 System Architecture

The system has three layers: the React frontend, the FastAPI backend, and the external AI services. Initially everything was written in a single `main.py` file, but it grew to nearly 700 lines and became hard to manage. I then split it into separate modules, each handling one specific concern. This made the code much cleaner and easier to work on. The diagram below shows how the modules are organised.

```
React Frontend
     |
     | HTTP (Axios)
     v
FastAPI (main.py) -- routes only, no business logic
     |
     +-- document_processor.py  (upload, chunking, embedding, agents
        )
     +-- chat.py                (Talk2Docs isolated chat sessions)
     +-- prompts.py             (all agent system prompts)
     +-- pdf_generator.py       (ReportLab PDF export)
     +-- analytics.py           (admin analytics queries)
     +-- database.py            (SQLAlchemy ORM, all DB helpers)
     +-- config_loader.py       (config.properties singleton)
     |
     +-- Qdrant (in-memory vector store, per-user collection)
     +-- Azure OpenAI (GPT-4 for generation, Ada-002 for embeddings)
     +-- SQLite (users, history, document library)
```

Listing 3.1: High-level architecture and module layout

3.2.1 Backend Module Structure

Module	Responsibility
main.py	HTTP routing only. No business logic — receives requests and delegates to modules.
document_processor.py	PDF parsing, chunking, embedding, Qdrant population, agent initialisation, analysis orchestration.
chat.py	Talk2Docs chatbot. Isolated ChatSession with its own ConversationBufferMemory.
prompts.py	All agent system prompts and task queries. Single source of truth for all AI behaviour.
pdf_generator.py	ReportLab branded PDF generation for analysis report export.
analytics.py	Admin analytics queries — by type, daily activity, top users, average duration.
database.py	SQLAlchemy ORM models and all DB helper functions including paginated history queries.
config_loader.py	Reads config.properties at startup. Exposes a typed AppConfig singleton.

Table 3.2: Backend module responsibilities

3.2.2 Frontend Component Structure

Component / Hook	Responsibility
App.js	Root — routing, SessionGuard wrapper, toast notifications.
Sidebar.js	API config, document upload, Upload/Library tabs, collapse with 52 px rail mode.
AnalysisPanel.js	Analysis type selection, Run button, results with ReactMarkdown, paginated history.
Talk2Docs.jsx	Floating chat button and drawer. Visible only after upload. Full conversation thread.
SessionTimeoutBanner.jsx	Warning banner with live countdown before auto-logout. Extend or Logout actions.
AdminPage.js	User management CRUD — create, edit, toggle active, delete.
AnalyticsPage.js	Platform analytics dashboard with charts.
useAuth.js	Auth context — login, logout, user state, session ID.
useSessionTimeout.js	Fetches config, detects activity, manages countdown, triggers auto-logout.
api.js	Centralised Axios client. All backend calls in one file. Base URL from env variable.

Table 3.3: Frontend component responsibilities

3.3 Document Ingestion and Chunking

When a user uploads a PDF, document_processor.py reads the text from each page using PyMuPDF. It then splits the text into smaller chunks using LangChain's RecursiveCharacterTextSplitter. The chunk size (1000 characters) and overlap (200 characters) are set in config.properties so they can be changed without touching the code. The overlap is there because without it, a sentence that falls at the boundary between two chunks gets cut off and loses its context. With 200 characters of overlap, the end of one chunk also appears at the start of the next, so the retriever can still find content that spans a boundary.

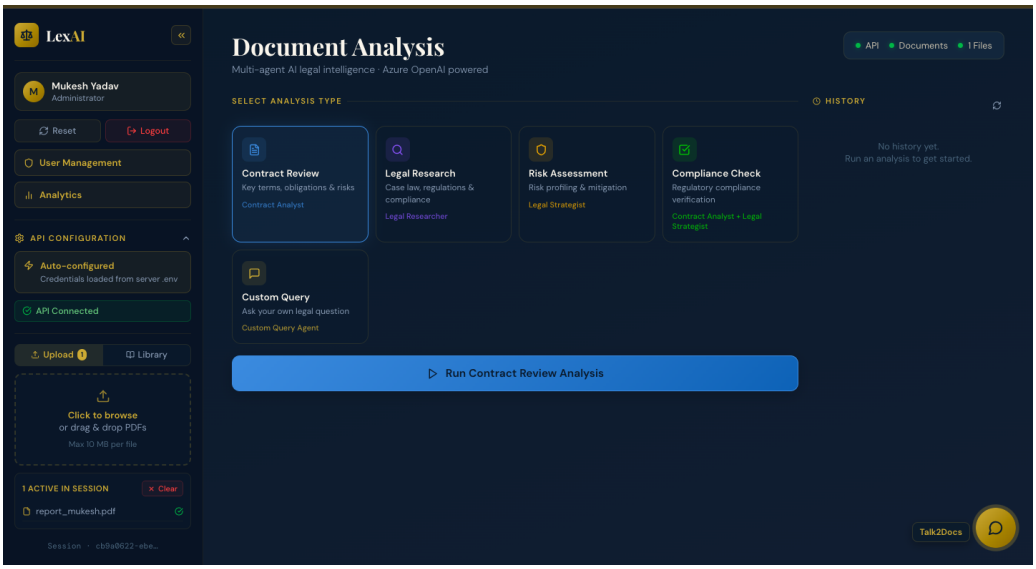


Figure 3.2: Sidebar Upload tab showing drag-and-drop zone, uploaded file list, and chunk count

3.4 Embedding and Vector Storage

Each chunk is converted into a 1536-dimensional vector using Azure OpenAI's `text-embedding-ada-002` model. These vectors are stored in a Qdrant in-memory collection that is specific to each user session. When an agent needs to find relevant parts of the document, it runs a cosine similarity search and gets back the top k matching chunks (default $k = 5$, configurable).

I chose vector search over keyword search because legal documents often say the same thing in different ways. For example, a search for "payment terms" would not find a clause that says "remuneration schedule" using keyword matching, but a vector search would find it because both phrases mean roughly the same thing and end up close to each other in the embedding space.

3.5 Multi-Agent Analysis Framework

Instead of using one big prompt to do everything, I split the analysis work across multiple agents where each agent has a specific job. Table 3.4 shows the five agents and what they do.

Agent	Used In	Primary Focus
Contract Analyst	Contract Review, Compliance Check	Clause-by-clause review, obligations, enforceability, missing provisions.
Legal Researcher	Legal Research	Statutes, regulations, case law, jurisdictional analysis.
Legal Strategist	Risk Assessment, Compliance Check	Risk register with severity scores, mitigation roadmap, negotiation points.
Custom Query Agent	Custom Query	Targeted Q&A with document-grounded evidence and clause citations.
Team Lead	All multi-agent types	Synthesises specialist outputs into a unified final report.

Table 3.4: Multi-agent pipeline — roles and analysis type assignments

Each agent has its own system prompt in `prompts.py` that defines its role, what sections it must cover, and what format to use for the output. For example, the Contract Analyst is told to always include the clause number, to cover six specific sections (Document Identification, Commercial Terms, Liability, Termination, Dispute Resolution, and Missing Provisions), and to flag if any of these sections are missing from the document.

I noticed early on that if the prompt just says what to analyse but does not say how to format the output, the agents give inconsistently formatted responses. Once I added explicit instructions about sections and headings, the output became much more consistent. For multi-agent analysis types, the specialist agents run one after another and their outputs are collected and passed to the Team Lead, which writes the final report. For Custom Query, only the one relevant agent runs.

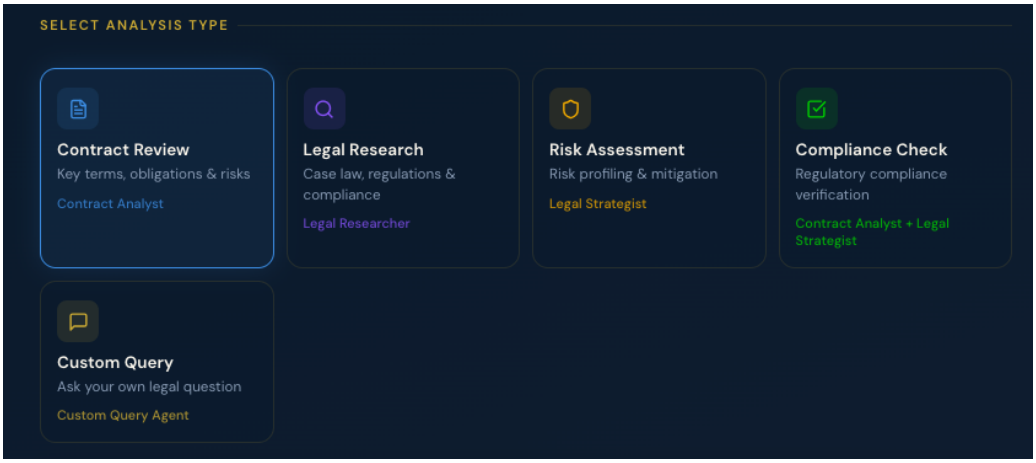


Figure 3.3: Analysis type selection cards and Run Analysis button

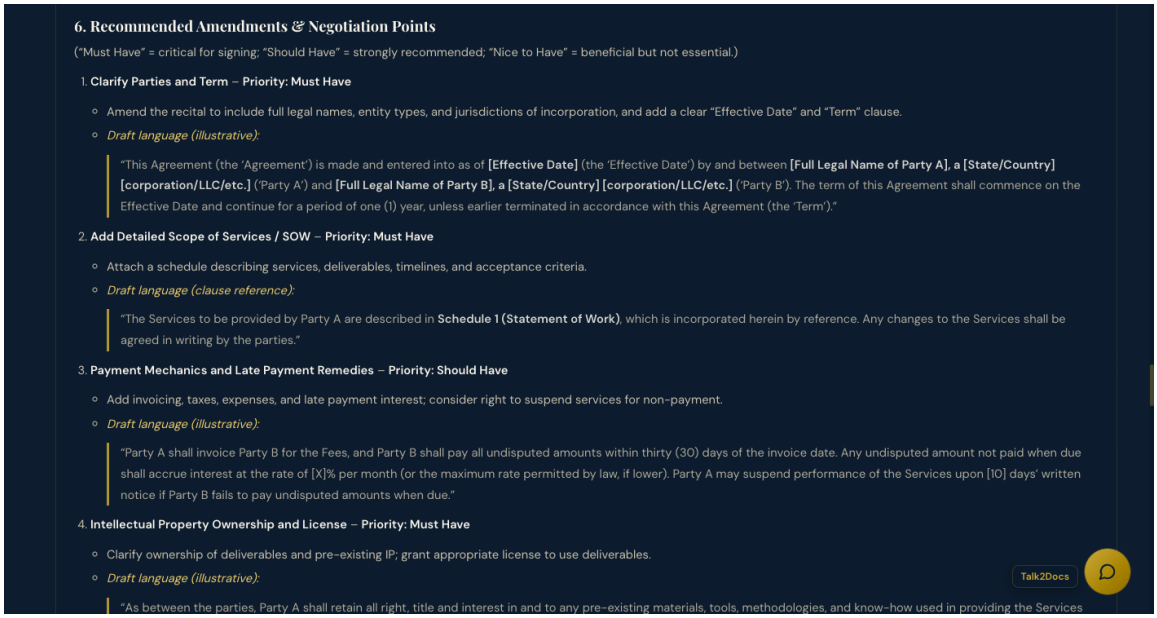


Figure 3.4: Multi-agent report output with markdown-rendered sections and clause citations

3.6 Prompts Architecture

All prompts are stored in one file, `prompts.py`, organised into three dictionaries:

- `AGENT_SYSTEM_PROMPTS` — the persona and instructions for each agent.
- `ANALYSIS_QUERIES` — the task prompt for each analysis type.
- `TALK2DOCS_SYSTEM_PROMPT` — the persona for the chat agent.

Keeping all prompts in one place makes it easy to update them and add new analysis types. Adding a new type only requires adding entries to these dictionaries and wiring up the right agents in `document_processor.py`.

3.7 Talk2Docs — Conversational Document Chat

Talk2Docs is a chat feature that lets users ask questions about the document in plain language. It uses the same RAG setup as the analysis agents. The chat button only appears after a document has been uploaded, so users cannot accidentally start chatting without any document loaded. Clicking the button opens a chat drawer where users can type questions and get answers with specific clause citations. The drawer also shows a typing indicator while a response is being generated, and it remembers the conversation history across multiple turns.

The chat sessions are kept completely separate from the analysis agents. Each user has their own `ChatSession` object in `chat.py` with its own memory. I found during testing that if the chat agent shared memory with the analysis agents, it would mix up the chat history with the analysis outputs and give confusing answers. Keeping them separate fixed this. When a user uploads a new document, `build_chat_agent()` is called again so the chat agent always uses the latest document.

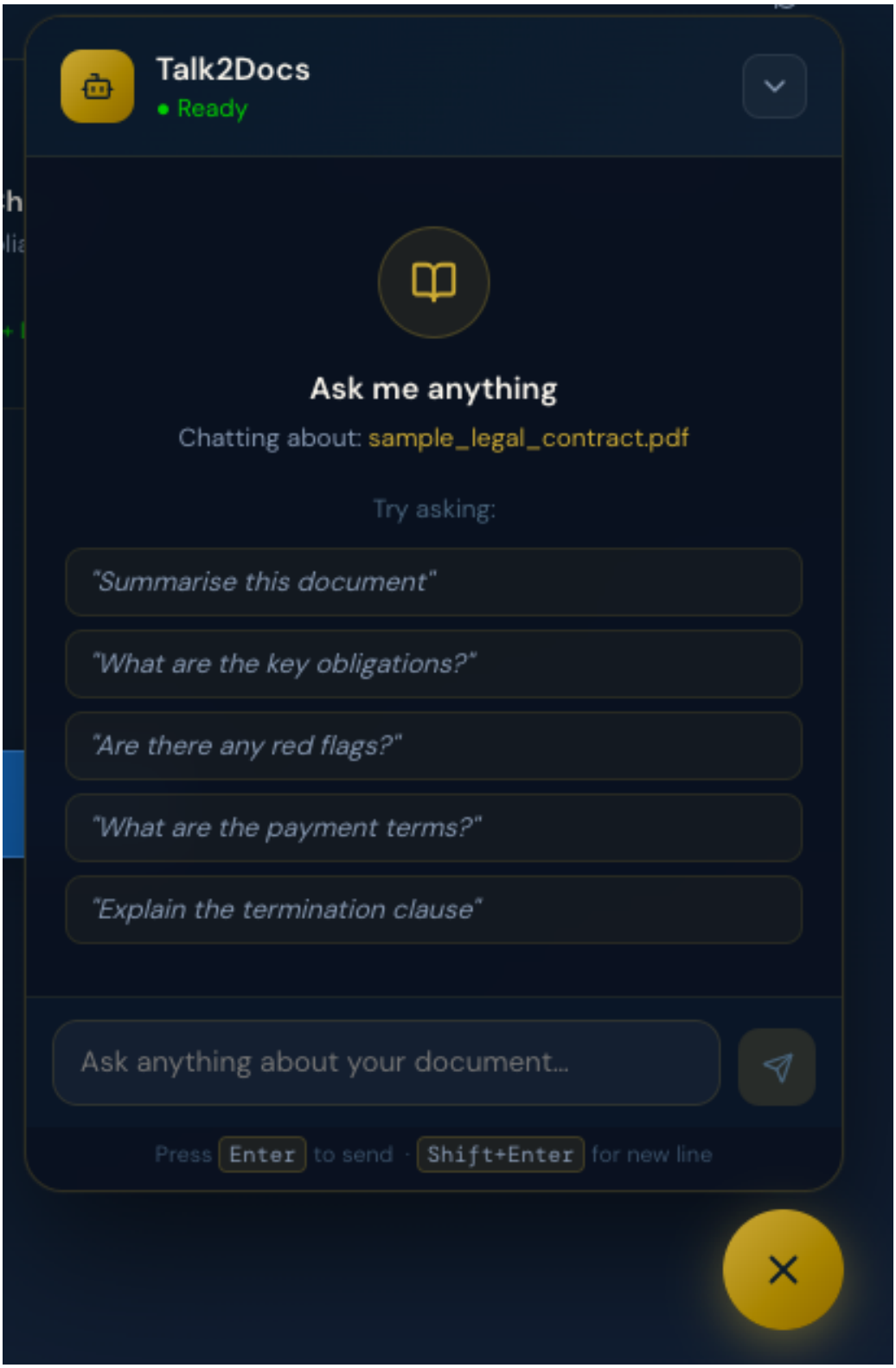


Figure 3.5: Talk2Docs floating button and chat drawer with active multi-turn conversation thread

3.8 Session Timeout and Configuration Management

All the settings for the system are stored in a file called `config.properties`. This file is read once when the server starts up by `config_loader.py`, which creates a single shared config object.

No module reads the file directly — they all use the shared object from `config_loader`. This means settings can be changed without modifying the source code.

The backend has an endpoint called `/app-config` that sends these settings to the frontend. The frontend uses them to manage the session timeout. If a user has been inactive for the set period (default is 30 minutes), a warning banner appears showing a countdown. If the user does something (mouse movement, click, key press) while the warning is showing, the timer resets. If the countdown reaches zero, the user is automatically logged out.

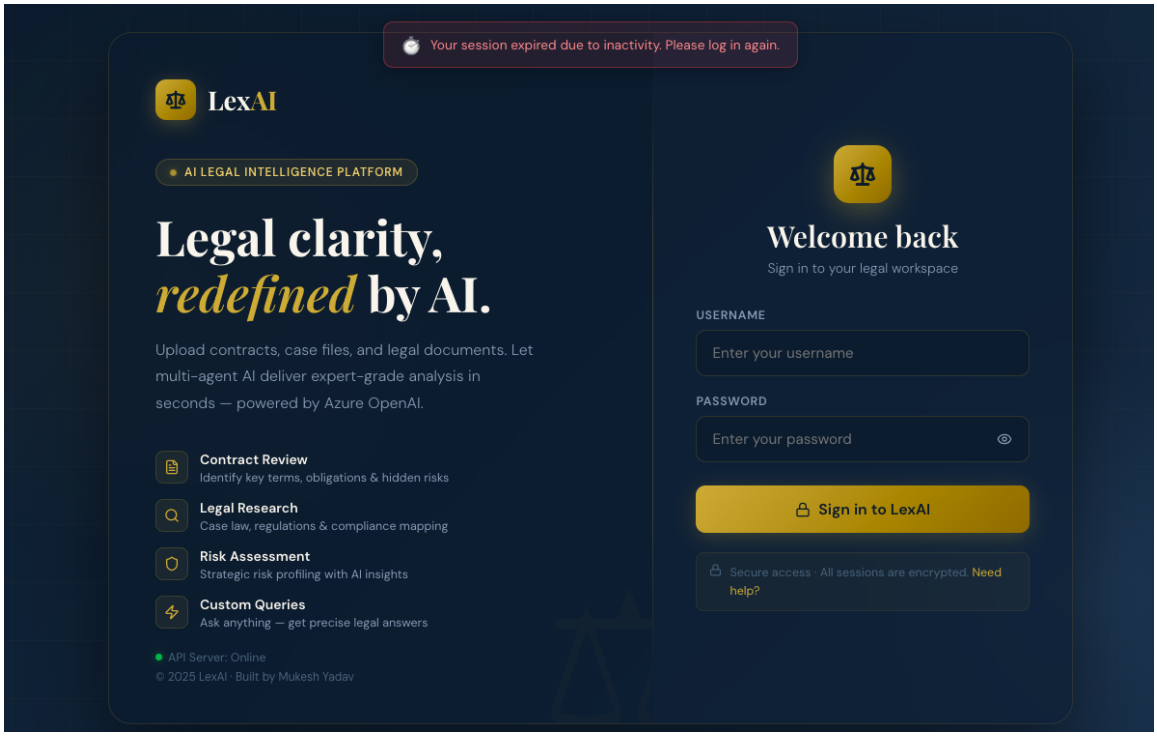


Figure 3.6: Session timeout warning banner with live countdown and Extend Session / Log Out buttons

3.9 API Endpoints

Method	Endpoint	Purpose
POST	/auth/login	User authentication
GET	/env-status	Check if server .env credentials are configured
POST	/configure	Configure Azure OpenAI credentials for session
POST	/configure-from-env	Auto-configure from server environment variables
POST	/upload-documents	Upload, chunk, embed, and store PDF files
POST	/analyze	Run multi-agent legal analysis
GET	/history/{username}	Paginated analysis history
DELETE	/history/{username}/{id}	Delete a history entry
GET	/documents/{username}	Document library listing
DELETE	/documents/{username}/{id}	Remove document from library
POST	/chat/{user_id}	Send message to Talk2Docs
GET	/chat/{user_id}/history	Retrieve conversation thread
DELETE	/chat/{user_id}	Clear conversation thread
POST	/export/pdf	Generate branded PDF report
GET	/analytics	Platform analytics (admin only)
GET	/app-config	Runtime config for frontend
CRUD	/admin/users	User management (admin only)

Table 3.5: LexAI REST API endpoints

3.10 Database Schema

Three tables are managed through SQLAlchemy ORM:

- **users:** `id`, `username`, `email`, `full_name`, `hashed_password (bcrypt)`, `role (admin/user)`, `is_active`, `created_at`
- **analysis_history:** `id`, `username`, `analysis_type`, `document_names`, `query`, `final_output`, `agent_outputs (JSON)`, `duration_secs`, `created_at`
- **documents:** `id`, `username`, `filename`, `file_size_kb`, `chunk_count`, `uploaded_at`

3.11 Configuration File

Key	Default	Effect
`session_timeout_minutes`	30	Inactivity duration in minutes before auto-logout
`session_warning_minutes`	5	How early to show the timeout warning banner
`activity_reset_enabled`	true	Whether user interaction resets the countdown
`chunk_size`	1000	Characters per document chunk
`chunk_overlap`	200	Overlap characters between adjacent chunks
`retriever_top_k`	5	Chunks returned per similarity search
`history_page_size`	10	Analysis history entries per page
`max_file_size_mb`	10	Maximum PDF upload size in MB
`analytics_daily_days`	30	Days of activity shown in analytics chart

Table 3.6: `config.properties` keys and their effects

3.12 Authentication and Security

- **Password hashing:** All passwords hashed with `bcrypt`. Plaintext passwords are never persisted or logged.
- **Session isolation:** Each user session has a UUID-based ID. Qdrant collections, agent instances, and chat sessions are all keyed by this ID — one user's documents cannot affect another's.
- **Role-based access:** Admin endpoints check the requesting user's role before processing.
- **Input validation:** FastAPI Pydantic models validate all incoming request payloads. Type errors and missing fields are rejected before reaching business logic.
- **Network access:** The backend binds to `0.0.0.0` to accept connections from any machine on the local network. The frontend reads `REACT_APP_API_URL` from a `.env` file so the IP address is never hardcoded in source.

3.13 Data Flow

3.13.1 Document Upload

1. User selects PDFs via the drag-and-drop upload zone in the sidebar.
2. Frontend sends a multipart POST to `/upload-documents` with the files and username.
3. `document_processor.py` extracts text from each page using PyMuPDF.
4. `RecursiveCharacterTextSplitter` splits the text into overlapping chunks using sizes from `config.properties`.
5. Each chunk is embedded by Azure Ada-002 and stored in the user's Qdrant collection.
6. LangChain analysis agents and the Talk2Docs agent are initialised with a retriever pointing to this collection.
7. The document record (filename, size, chunk count) is saved to the `documents` table in SQLite.
8. Frontend updates state: `documentsUploaded = true`. The Talk2Docs button becomes visible.

3.13.2 Analysis Pipeline

1. User selects an analysis type and clicks Run Analysis.
2. The sidebar auto-collapses to the 52 px rail, giving the analysis panel full browser width.
3. Frontend sends POST `/analyze`.
4. `document_processor.py` identifies which specialist agents to invoke for this analysis type.
5. Each agent calls the `document_search` tool — cosine similarity query returns the top k chunks.
6. Azure GPT-4 generates each agent's structured output grounded in the retrieved chunks.

7. For multi-agent types, all specialist outputs are passed to the Team Lead for synthesis.
8. The final report is saved to `analysis_history` in SQLite.
9. The frontend renders the markdown report. PDF export and history save become available.

3.13.3 Talk2Docs Chat

1. User types a message in the Talk2Docs drawer.
2. Frontend sends POST `/chat/{user_id}`.
3. `chat.py` retrieves the user's `ChatSession` with its isolated `ConversationBufferMemory`.
4. The LangChain agent invokes `document_search` to retrieve relevant document passages.
5. GPT-4 receives the system prompt, full conversation history, and retrieved chunks.
6. A conversational reply is generated with specific clause citations.
7. The reply is appended to the thread and returned as JSON. ReactMarkdown renders it in the drawer.

Chapter 4

Results and Discussions

4.1 Features Implemented

4.1.1 Five Analysis Modes

- **Contract Review:** Clause-by-clause analysis covering obligations, liability, termination, dispute resolution, and missing provisions.
- **Legal Research:** Statutory and regulatory identification, case law references, and jurisdictional analysis.
- **Risk Assessment:** Risk register with severity and likelihood scoring, mitigation roadmap, and negotiation points.
- **Compliance Check:** Framework identification, clause-by-clause compliance audit, non-compliance findings, and risk quantification.
- **Custom Query:** User-defined question answered with document-grounded evidence and specific clause citations.

4.1.2 Document Library with Upload/Library Tabs

The sidebar has two tabs. The **Upload tab** shows files that have been loaded into the current session. There is a Clear button that removes them from the vector store but does not delete the record from the database. The **Library tab** shows all files the user has ever uploaded, pulled from the database. This way users can keep track of what they have analysed before even after clearing a session.

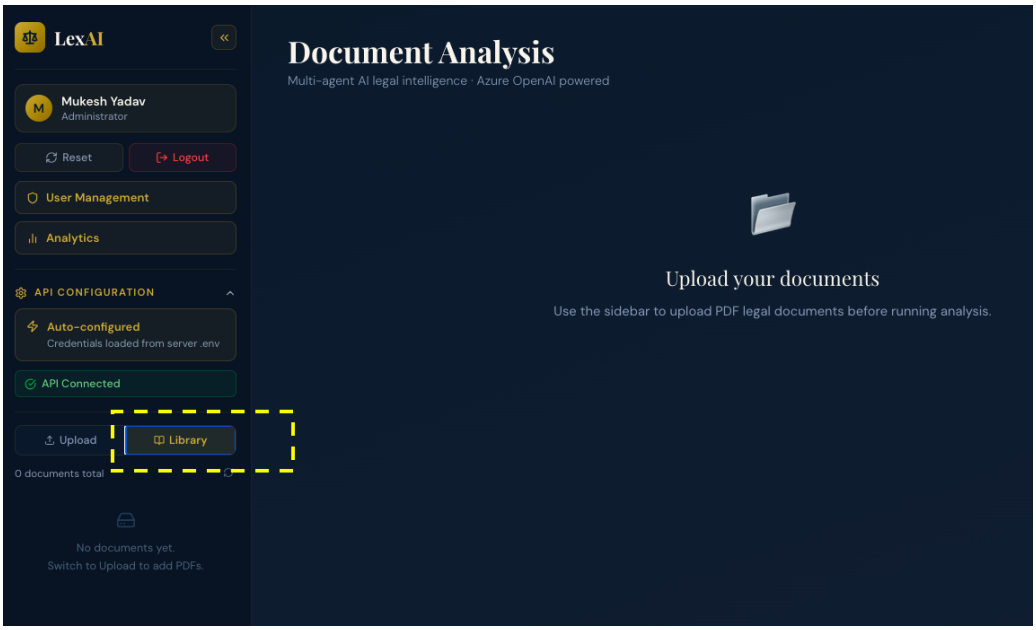


Figure 4.1: Sidebar Upload tab and Library tab showing active session files and all-time document history

4.1.3 Collapsible Sidebar with Rail Mode

When the sidebar is open it takes up 280 px of width. When the user clicks Run Analysis, the sidebar automatically collapses to a 52 px strip showing just icons, so the analysis panel gets the full screen width. Users can manually expand or collapse the sidebar at any time using the collapse button.

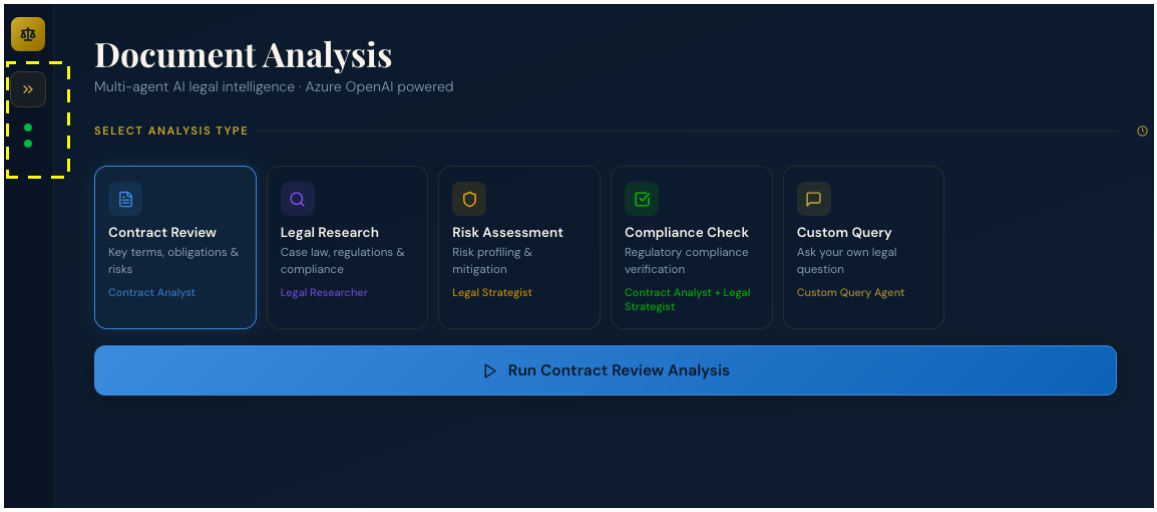


Figure 4.2: Full sidebar state vs collapsed rail state showing status indicator and expand button

4.1.4 Paginated Analysis History

Every completed analysis is saved to SQLite and shown in the history panel with 10 entries per page (configurable). Each entry shows the analysis type, document name, timestamp, and duration. Entries can be opened to view the full report, exported to PDF, or deleted.

4.1.5 Admin Panel and Analytics

Admin users access a User Management page (create, edit, toggle active, delete users) and an Analytics page showing: total analyses, breakdown by analysis type, daily activity chart for the past 30 days, average analysis duration, and top active users. All parameters are configurable via `config.properties`.

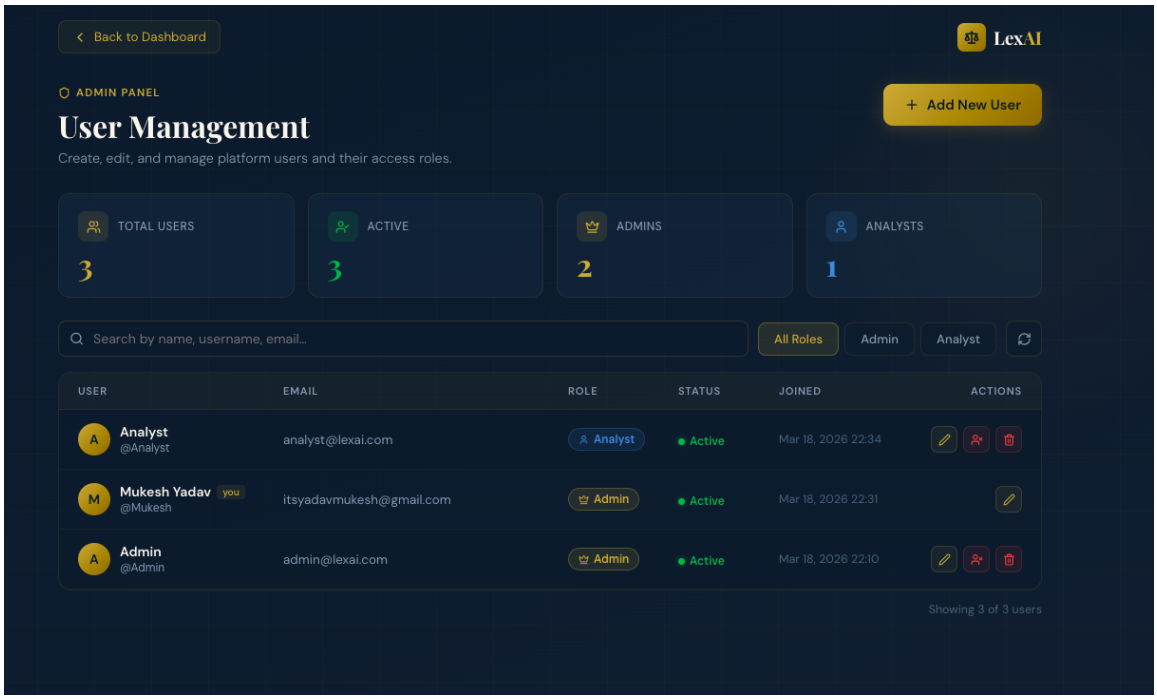


Figure 4.3: Admin analytics dashboard showing analysis type breakdown and daily activity trend

4.1.6 Branded PDF Export

Any analysis result can be exported as a branded PDF generated server-side by ReportLab. The report includes a cover page, full analysis content, per-agent detail sections, and a branded footer. The PDF is streamed directly to the browser and downloaded automatically.

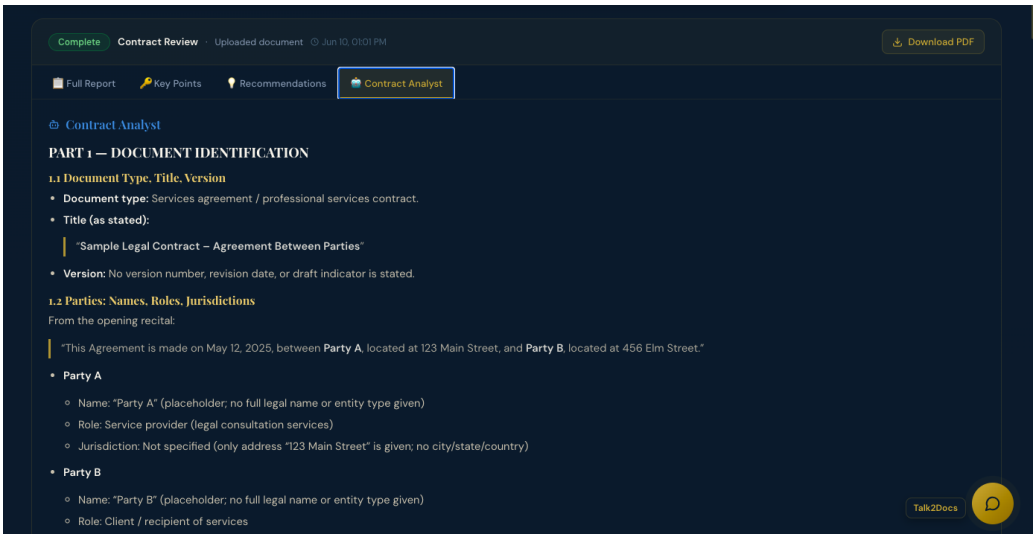


Figure 4.4: Exported PDF report showing cover page and analysis content

4.2 Evaluation

4.2.1 Test Document

To test the system properly I created a six-page sample SaaS Subscription Agreement that had several problematic clauses built into it on purpose. These included an auto-renewal clause with a narrow 60-to-90-day cancellation window, compound monthly interest on late payments, a liability cap that only applies to the vendor (the client's liability was stated as unlimited), a clause giving the vendor a permanent licence to use the client's data for AI training, and penalty clauses for things like reverse engineering and benchmarking.

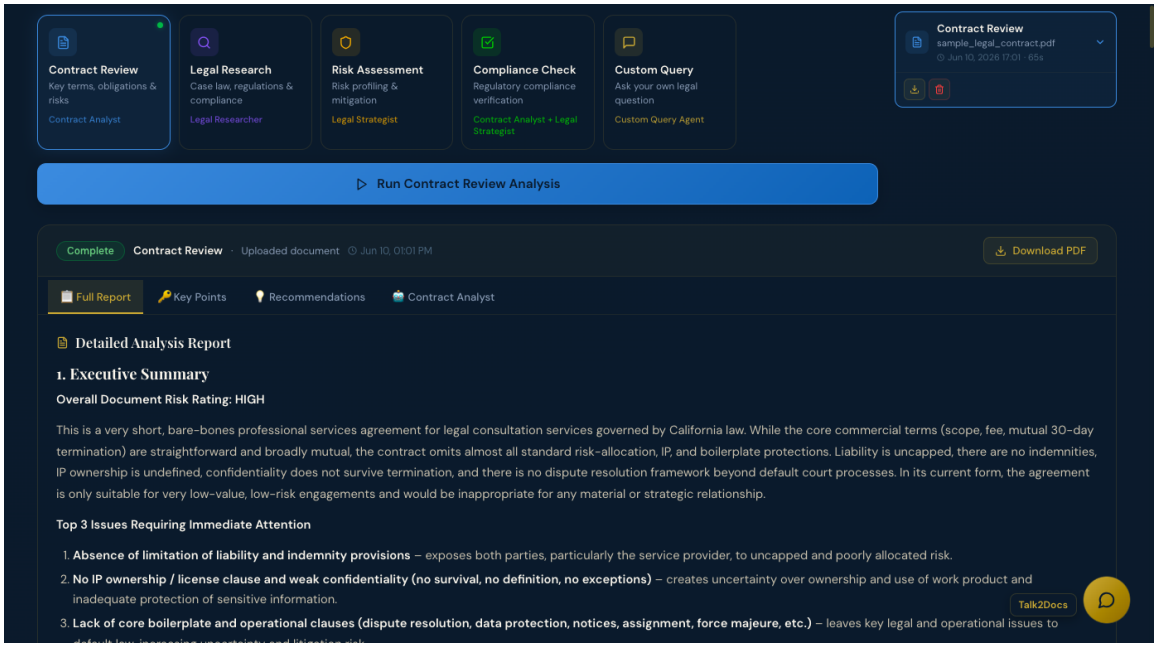


Figure 4.5: Test contract PDF uploaded to LexAI alongside the generated analysis output

4.2.2 Qualitative Observations

- **Risk Assessment:** The Legal Strategist identified and flagged 12 material risk provisions including the liability asymmetry, compound interest clause, and narrow non-renewal window. Each finding cited the specific clause number.
- **Contract Review:** The Contract Analyst covered all six required output sections with specific clause citations throughout.
- **Custom Query:** "What happens if I miss the cancellation deadline?" returned a precise answer citing Clause 4.2 and explaining that notice outside the 60-to-90-day window is void and the agreement renews automatically.
- **Talk2Docs:** Multi-turn conversation maintained context correctly across multiple messages. "Can they use my data to train their AI?" returned a correct answer citing Clause 5.3 with the specific licence language and the absence of any opt-out provision.

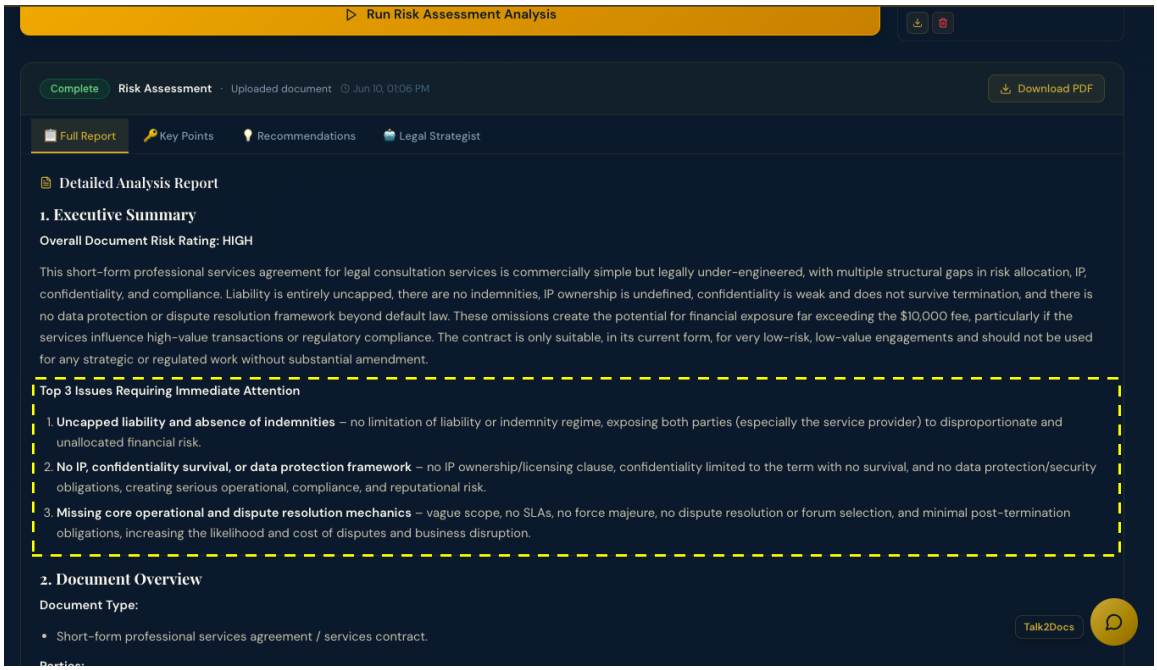


Figure 4.6: Risk Assessment output showing flagged clauses with severity and clause citations

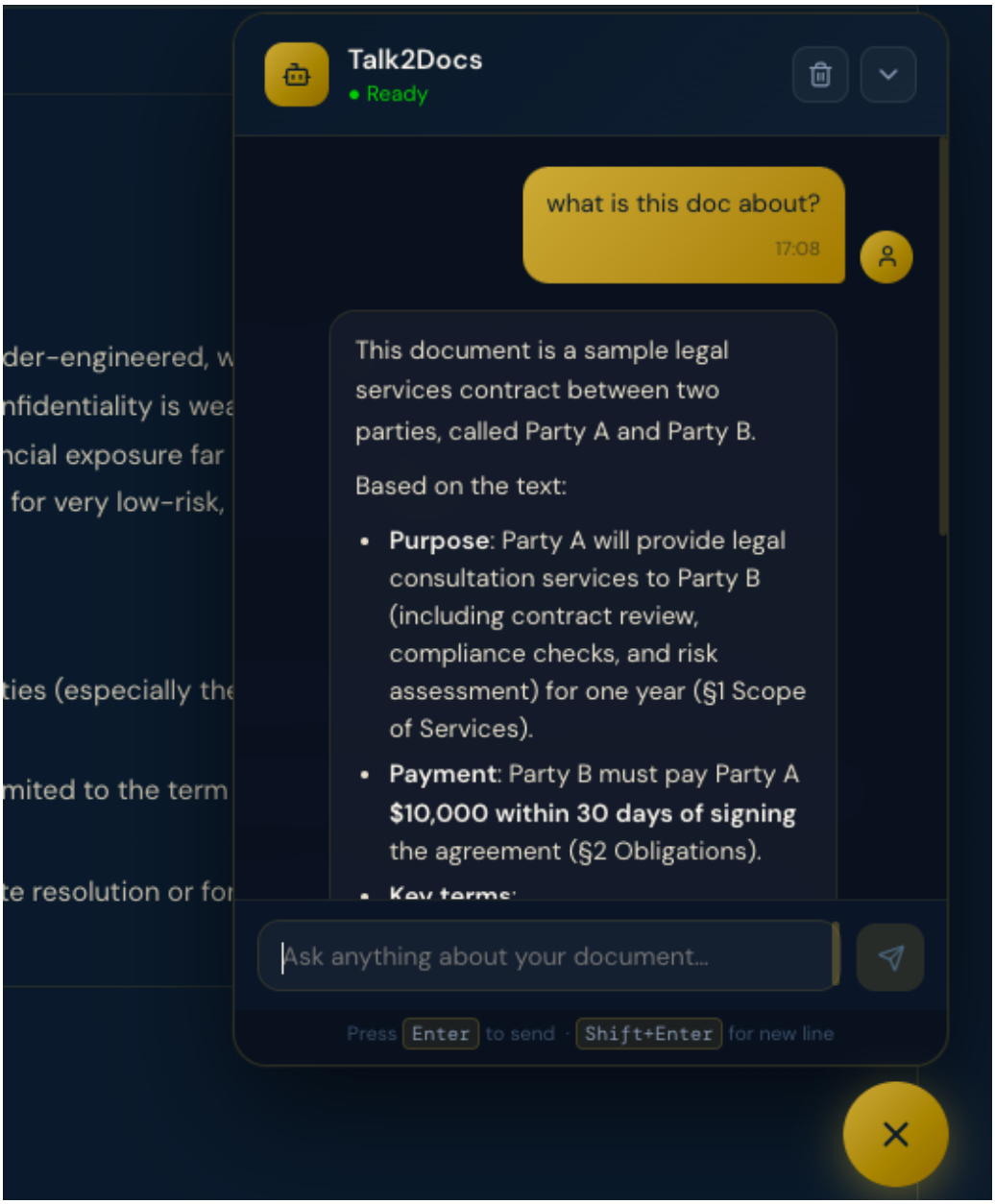


Figure 4.7: Talk2Docs answering a question with specific clause citation

4.2.3 Limitations

- **In-memory vector store:** Qdrant runs in memory and is lost on server restart. Documents must be re-uploaded. A persistent Qdrant instance would address this.
- **No response streaming:** Responses return as complete JSON objects. Longer analyses require the user to wait without incremental feedback.
- **Single document session:** Comparing two contracts simultaneously is not supported in the current architecture.
- **Mobile layout:** Fixed-width sidebar and two-column analysis panel do not adapt to narrow screens without a responsive redesign.
- **Token limits:** Very long documents may approach the LLM context window limit when multiple large chunks are retrieved together.

Chapter 5

Conclusions

5.1 Challenges and Learnings

5.1.1 Multi-Agent Memory Isolation

One issue I ran into early was that the analysis agents and the Talk2Docs chat agent were sharing the same memory object. This caused a problem where asking a question in the chat would mix up the chat history with earlier analysis outputs. For example, asking about payment terms would sometimes produce a response that referenced a full Contract Review that had been run earlier, rather than searching the document fresh. The fix was simple — give each agent type its own memory object in its own session class. But understanding why it was happening required learning how LangChain handles memory and why having shared mutable state across agents causes problems.

5.1.2 Consistent Agent Output Format

In the early versions of the prompts, the agents gave well-written responses but the formatting kept changing between runs. Sometimes the output used bullet points, sometimes numbered lists, and the section headings were different each time. ReactMarkdown could display all of these, but it looked messy and unprofessional. I fixed this by adding specific instructions to each task prompt telling the agent exactly which sections to include and in what order. The main learning here was that when writing prompts for structured output, you need to be as specific about the format as you would be when writing a function specification in code.

5.1.3 Backend Modularisation

As mentioned earlier, `main.py` grew to around 700 lines before I split it up. The split itself was not difficult but it forced me to think carefully about which parts of the code share state and which should be independent. The trickiest part was the `config_loader` module. It needs to read the config file exactly once when the server starts and then share that object with all other modules. Getting this to work without causing circular imports took a few tries.

5.1.4 Session Timeout Coordination

The session timeout feature was harder to implement than it looks because it involves four separate parts of the system working together: the backend holds the config, the auth hook handles logout, the React router handles navigation, and the timeout hook runs the countdown. Getting all four to work together without circular dependencies or React hook issues took several attempts. The final solution was a `useSessionTimeout` hook that fetches the config from the backend and exposes only the specific values that the UI components need.

5.2 Security, Ethics and Compliance

- **Not legal advice:** LexAI is explicitly an analytical assistant, not a substitute for qualified legal advice. Every analysis output includes a disclaimer to this effect.
- **Data handling:** Documents are processed in memory and not permanently stored on disk beyond the original upload. The vector store is in-memory and cleared on session end or server restart.
- **Password security:** All passwords hashed with bcrypt. Plaintext passwords are never written to disk or logged.
- **Session security:** Auto-logout after configurable inactivity. Session IDs are UUID-based and non-guessable.
- **AI transparency:** Every analysis cites the specific clause supporting each finding, allowing users to verify every claim against the source document.
- **No third-party tracking:** No external analytics or advertising systems are integrated. Platform metrics are stored only in the local SQLite database.

5.3 Future Work

There are several improvements I would like to make to the system in the future. The biggest one is switching to a persistent Qdrant instance so that uploaded documents do not need to be re-uploaded every time the server restarts. Another useful improvement would be streaming responses, so users can see the output being generated rather than waiting for the full response. Supporting multiple documents at the same time (for example, comparing two contracts side by side) is also something the current architecture could support with some changes to how retrieval is scoped. On the frontend side, the layout does not work well on mobile screens and would need a responsive redesign to work on phones. Finally, replacing the Azure OpenAI API with a smaller open-source model fine-tuned on legal data like CUAD could reduce costs and remove the dependency on external APIs for users who have privacy concerns.

5.4 Conclusion

This project showed that using a multi-agent RAG setup gives much better results for legal document analysis than using a single LLM prompt. By giving each agent a specific role and its own detailed prompt, and by using vector search to retrieve the relevant parts of the document before generating any output, the system can cover a contract from multiple angles in one run. The retrieval step also ensures that every finding points back to a specific clause in the actual document, which reduces the risk of the model making things up.

Building this project also taught me that a working AI pipeline is only part of what makes a useful system. Features like session timeout, document history, PDF export, the admin panel, and the Talk2Docs chat all required careful thought about what a real user would actually need. Getting the balance right between the AI parts and the product parts was one of the more interesting challenges in this project.

The system has real limitations — the in-memory vector store, the lack of streaming, and the single-document restriction are all things that could be improved. But the core functionality is complete and works as intended. A user can upload a legal document, get a structured multi-perspective analysis with specific clause citations, ask follow-up questions, and download the result as a PDF. For the target audience of students, freelancers, and small businesses, that

is a genuinely useful tool.

Bibliography

[1] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] S. Yao, J. Zhao, D. Yu, et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *International Conference on Learning Representations (ICLR)*, 2023.

[3] J. Wei, X. Wang, D. Schuurmans, et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] D. Hendrycks, C. Burns, A. Chen, and S. Ball, "CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review," *arXiv preprint arXiv:2103.06268*, 2021.

[5] Y. Koreeda and C. D. Manning, "ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts," *Findings of EMNLP*, 2021.

[6] I. Chalkidis, A. Jana, D. Hartung, et al., "LexGLUE: A Benchmark Dataset for Legal Language Understanding in English," *Proceedings of ACL*, 2022.

[7] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, "LEGAL-BERT: The Muppets Straight Out of Law School," *Findings of EMNLP*, 2020.

[8] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.

[9] K. D. Ashley, *Artificial Intelligence and Legal Analytics: New Tools for Law Practice in the Digital Age*. Cambridge University Press, 2017.

[10] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking Large Language Models in Retrieval-Augmented Generation," *arXiv preprint arXiv:2309.01431*, 2023.

[11] LangChain, "Agent Framework and Tools," `https://python.langchain.com/docs/modules/agents/`, 2024.

[12] Microsoft, "Azure OpenAI Service Documentation," `https://learn.microsoft.com/en-us/azure/ai-services/openai/`, 2024.

[13] Qdrant, "Vector Database Documentation," `https://qdrant.tech/documentation/`, 2024.

[14] S. Ramirez, "FastAPI Documentation," `https://fastapi.tiangolo.com/`, 2024.

[15] ReportLab Inc., "ReportLab PDF Library," `https://www.reportlab.com/`, 2024.

[16] Meta Open Source, "React Documentation," `https://react.dev/`, 2024.

[17] "SQLAlchemy ORM Documentation," `https://docs.sqlalchemy.org/`, 2024.

[18] "PyMuPDF Documentation," `https://pymupdf.readthedocs.io/`, 2024.

[19] "Pydantic Data Validation Documentation," `https://docs.pydantic.dev/`, 2024.